Program 5 KNN

```
import random as rd
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score

X_list =[rd.random() for i in range(0,100)]
Y_list = [0]*len(X_list)
for i in range(len(X_list)):
    if X_list[i] <= 0.5:
        Y_list[i] = "Class1"
    else:
        Y_list[i] = "Class2"



X = np.array(X_list).reshape(-1,1) #KNN model takes 2D array
Y = np.array(Y_list)

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=
0.5, train_size= 0.5, shuffle= False, stratify=None)
print(f'Number of training point: {len(X_train)}\n')
print(f'Number of test point: {len(X_test)}\n')

K = [1,2,3,4,5,20,30]
acc,cm = [],[]
for i in K:
    classify = KNeighborsClassifier(n_neighbors=i)
    classify.fit(X_train, Y_train)
    Y_predict = classify.predict(X_test)

    cm_i = confusion_matrix(Y_test, Y_predict)
    cm.append(cm_i)
    acc_i = accuracy_score(Y_test, Y_predict)
    acc.append(acc_i)
    print(f'For K={i}:\nconfusion
matrix:\n{cm_i}\naccuracy:{acc_i}')
plt.plot(K, acc)
plt.xlabel('No of Neighbours')
plt.ylabel('Accuracy')
```

```
print(f'For K={i}:\nconfusion matrix:\n{cm_i} \naccuracy:{acc_i}')
plt.plot(K, acc)
plt.ylim(0.8, 1.1)
```

Program 6 LWR

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load the dataset using pandas
url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/auto-mpg/auto-mpg.data"
column_names = ['MPG', 'Cylinders', 'Displacement', 'Horsepower',
'Weight', 'Acceleration', 'Model Year', 'Origin', 'Car Name']
df = pd.read_csv(url, names=column_names, na_values='?',
comment='\t', sep=' ', skipinitialspace=True)
df = df.drop('Car Name', axis =1)
df = df.dropna()

# Prepare the data
X = np.array(df.Weight).reshape(-1,1)
y = np.array(df.MPG)

# Scale X values
X = StandardScaler().fit_transform(X)

# Model
model = LinearRegression()

y_p = [0]*len(X)
tau = 0.1 # Bandwidth parameter

for i in range(len(X)):
    w = [0]*len(X)
    # Weights
    for j in range(len(X)):
        w[j]= np.exp(-((X[i]-X[j])**2)/(2*tau))
    w = np.array(w).reshape(-1,)
    model.fit(X,y, sample_weight = w)
```

```
    # Prediction for each point
    X_p = X[i].reshape(1,-1)
    y_p[i] = model.predict(X_p)

print ('Mean squared error: %.2f' % mean_squared_error(y, y_p))
print ('Coefficient of determination: %.2f' % r2_score(y, y_p)) # 1
is perfect prediction
# Plot Output
plt.scatter(X, y)
plt.scatter(X,y_p, color = 'red')
plt.xlabel('Independent variable')
plt.ylabel('Dependent variable')
plt.title('Locally Weighted Regression')
plt.show()
```

Program 7 LR and PR

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import fetch_california_housing

# Load the dataset using pandas
url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/auto-mpg/auto-mpg.data"
column_names = ['MPG', 'Cylinders', 'Displacement', 'Horsepower',
'Weight', 'Acceleration', 'Model Year', 'Origin', 'Car Name']
df = pd.read_csv(url, names=column_names, na_values='?',
comment='\t', sep=' ', skipinitialspace=True)
df = df.drop('Car Name', axis =1)
df = df.dropna()

# Prepare the data
X = np.array(df.Displacement).reshape(-1,1)
y = np.array(df.MPG)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Create linear regression object
```

```
model = LinearRegression()

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
print('Coefficients:', model.coef_)
print('Mean squared error: %.2f' % mean_squared_error(y_test,
y_pred))
print('Coefficient of determination: %.2f' % r2_score(y_test,
y_pred)) # 1 is perfect prediction

# Plot outputs
plt.scatter(X_test, y_test)
plt.plot(X_test, y_pred, color='red')
plt.xlabel('Independent variable')
plt.ylabel('Dependent variable')
plt.title('Linear Regression')
plt.show()


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score

# Load the dataset using pandas
url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/auto-mpg/auto-mpg.data"
column_names = ['MPG', 'Cylinders', 'Displacement', 'Horsepower',
'Weight', 'Acceleration', 'Model Year', 'Origin', 'Car Name']
df = pd.read0_csv(url, names=column_names, na_values='?',
comment='\t', sep=' ', skipinitialspace=True)
df = df.drop('Car Name', axis =1)
df = df.dropna()

# Prepare the data
```

```
X = np.array(df.Displacement).reshape(-1,1)
y = np.array(df.MPG)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Fit a polynomial to the feature
poly = PolynomialFeatures(2)
X_test_p = poly.fit_transform(X_test)
X_train_p = poly.fit_transform(X_train)

# Train a linear regression model
model = LinearRegression()
model.fit(X_train_p, y_train)

# Make predictions
y_pred = model.predict(X_test_p)

# Evaluate the model
print('Coefficients:', model.coef_)
print('Mean squared error: %.2f' % mean_squared_error(y_test,
y_pred))
print('Coefficient of determination: %.2f' % r2_score(y_test,
y_pred))
# 1 is perfect prediction

# Plot output
plt.scatter(X_test, y_test)
plt.scatter(X_test, y_pred, color='red')
plt.xlabel('Independent variable')
plt.ylabel('Dependent variable')
plt.title('Polynomial Regression')
plt.show()
```